

Lab 3: การใช้งาน Mininet เพื่อทดสอบประสิทธิภาพเครือข่ายเบื้องต้น

20 กรกฎาคม พ.ศ. 2569

มินิเน็ต (Mininet) เป็นโปรแกรมที่ทำงานบนระบบปฏิบัติการลินุกซ์ สำหรับจำลองเน็ตเวิร์คแบบอีมูเลเตอร์ (emulator) สามารถจำลองการทำงานของเน็ตเวิร์คด้วยการสร้างเวอร์ชวลไฮสตรัส สวิตช์ และลิงค์ โดยมินิเน็ตจะสร้างอินเตอร์เฟซ เรียกว่า เวอร์ชวล อีเทอร์เน็ต (virtual Ethernet, veth) เพื่อเชื่อมต่ออุปกรณ์เข้าด้วยกัน ทำหน้าที่เสมือนพอร์ตของ อีเทอร์เน็ตจริง ผู้ใช้สามารถกำหนดความเร็ว (speed) และเวลาหน่วง (delay) ของแพ็กเก็ตบนลิงค์ที่เชื่อมต่อได้ นอกจากนี้ ไฮสตรัสที่สร้างขึ้นในมินิเน็ตสามารถเรียกใช้โปรแกรม ต่าง ๆ ในระบบได้ผ่านทาง Secure Shell (SSH)

ในการทดลองนี้ นักศึกษาจะใช้งานมินิเน็ตเพื่อจำลองและทดสอบ **ประสิทธิภาพพื้นฐาน ของระบบเครือข่าย** ได้แก่ ความหน่วง (Delay) ความแปรปรวนของความหน่วง (Jitter) การสูญหายของแพ็กเก็ต (Packet Loss) และทราฟฟิค (Throughput) เพื่อให้เข้าใจว่าปัจจัย เหล่านี้ส่งผลต่อคุณภาพการสื่อสารอย่างไร ก่อนนำไปประยุกต์ใช้กับการวิเคราะห์เครือข่าย จริงในบทต่อไป

1 วัตถุประสงค์การเรียนรู้

เมื่อนักศึกษาทำการทดลองนี้เสร็จสิ้น นักศึกษาจะสามารถ

1. อธิบายความหมายและที่มาของค่า Delay, Jitter, Packet Loss, Bandwidth และ Throughput ได้อย่างถูกต้อง
2. ใช้งานมินิเน็ตเพื่อจำลองเน็ตเวิร์คและปรับแต่งพารามิเตอร์ของลิงค์ด้วยคำสั่ง `tc/netem`
3. วัดและบันทึกค่าประสิทธิภาพของเน็ตเวิร์คด้วยคำสั่ง `ping` และโปรแกรม `iperf3`
4. วิเคราะห์ความสัมพันธ์ระหว่าง Delay, Jitter, Loss กับ Throughput ของ TCP โดยอ้างอิงหลักการทางทฤษฎี
5. เปรียบเทียบผลการทดลองที่ได้จากการจำลองบนมินิเน็ตกับการทดสอบบนอุปกรณ์จริง

2 ทฤษฎีพื้นฐาน: ประสิทธิภาพของเครือข่าย

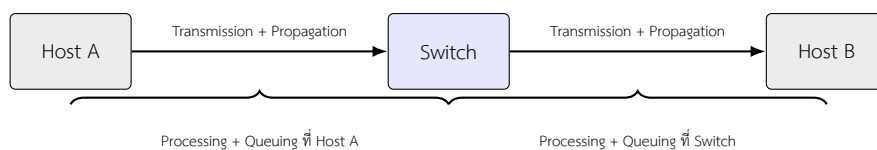
ก่อนเริ่มการทดลอง นักศึกษาควรเข้าใจความหมายของพารามิเตอร์ที่จะทดสอบ ดังนี้

2.1 ความหน่วง (Delay) และองค์ประกอบของความหน่วง

ความหน่วงปลายทางถึงปลายทาง (end-to-end delay) ประกอบด้วยความหน่วงย่อยหลายส่วน ดังแสดงในรูปที่ 1

- **Processing Delay** เวลาที่โหนดใช้ในการประมวลผลแพ็คเกจของแพ็กเก็ต
- **Queuing Delay** เวลาที่แพ็กเก็ตต้องรอในคิวก่อนถูกส่งออก ขึ้นกับปริมาณ ทราฟฟิกที่คับคั่งในขณะนั้น
- **Transmission Delay** เวลาที่ใช้ในการส่งข้อมูลทั้งหมดออกทางลิงค์คำนวณจาก L/R โดย L คือขนาดแพ็กเก็ต (bit) และ R คือ Bandwidth (bit/s)
- **Propagation Delay** เวลาที่สัญญาณเดินทางผ่านตัวกลาง คำนวณจาก d/s โดย d คือระยะทาง และ s คือความเร็วของสัญญาณในตัวกลาง

$$\text{ความหน่วงรวม} = \sum (\text{Processing} + \text{Queuing} + \text{Transmission} + \text{Propagation})$$

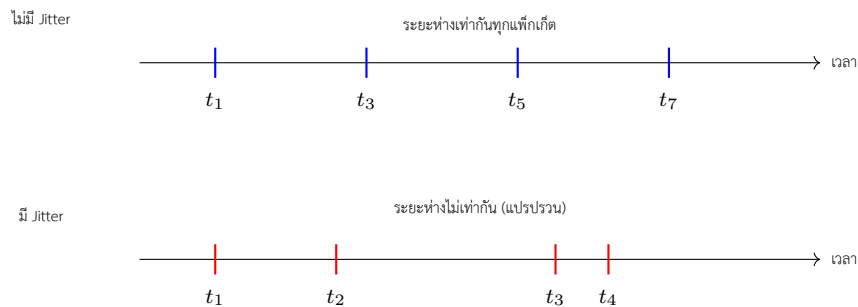


รูปที่ 1: องค์ประกอบของความหน่วงปลายทางถึงปลายทาง (End-to-end Delay)

ในการทดลองด้วยมินิเน็ตคำสั่ง `tc/netem` จะเพิ่มค่า **Transmission/ Propagation Delay** เทียมเข้าไปบนอินเตอร์เฟซที่กำหนด เพื่อจำลองสถานการณ์ที่โหนด อยู่ห่างกันทางภูมิศาสตร์ หรือลิงค์มีคุณภาพต่ำ

2.2 ความแปรปรวนของความหน่วง (Jitter)

Jitter คือความแปรปรวนของค่า Delay ในแต่ละแพ็กเก็ตที่ส่งออกไป ดังแสดงในรูปที่ 2 หากค่า Delay คงที่ทุกแพ็กเก็ต (กรณีอุดมคติ) ผู้รับจะสามารถเล่นข้อมูลได้อย่างต่อเนื่อง แต่หากมี Jitter สูง แพ็กเก็ตจะมาถึงไม่สม่ำเสมอ ซึ่งส่งผลกระทบต่อแอปพลิเคชันแบบเรียลไทม์ เช่น VoIP หรือวิดีโอคอล



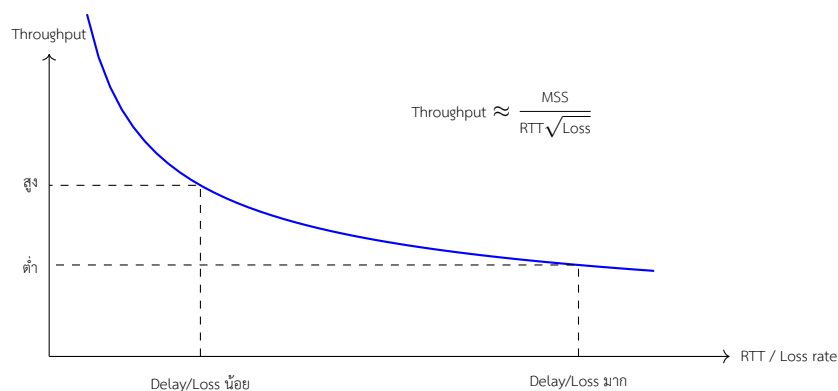
รูปที่ 2: แนวคิดของ Jitter: ความแปรปรวนของช่วงเวลาระหว่างแพ็กเก็ตที่มาถึงปลายทาง

2.3 การสูญหายของแพ็กเก็ต (Packet Loss)

Packet Loss คือสัดส่วนของแพ็กเก็ตที่ส่งออกไปแต่ไม่สามารถไปถึงปลายทางได้ สาเหตุหลักเกิดจาก Buffer overflow ที่อุปกรณ์กลางทาง, ข้อผิดพลาดของสัญญาณ (bit error) ที่ทำให้ต้องทิ้งแพ็กเก็ต, หรือ Congestion บนเส้นทาง สำหรับ TCP การสูญหายของแพ็กเก็ต จะกระตุ้นกลไก Congestion Control ให้ลดขนาด Congestion Window (Cwnd) ลง ส่งผลให้ Throughput ลดลงตามไปด้วย

2.4 Bandwidth และ Throughput

Bandwidth คือความสามารถสูงสุดทางทฤษฎีที่ลิงค์รองรับได้ (bit/s) ในขณะที่ **Throughput** คืออัตราการส่งข้อมูลจริงที่วัดได้ ซึ่งจะน้อยกว่าหรือ เท่ากับ Bandwidth เสมอ เนื่องจากมี Overhead ของโปรโตคอล, Delay, Jitter และ Packet Loss เข้ามาเกี่ยวข้อง ความสัมพันธ์นี้แสดงในรูปที่ 3



รูปที่ 3: ความสัมพันธ์ระหว่าง Delay/Loss กับ Throughput ของ TCP (สูตรประมาณของ Mathis et al.)

จากสูตรในรูปที่ 3 จะเห็นว่า Throughput ของ TCP แปรผกผัน กับ RTT (Round-Trip Time) และรากที่สองของอัตราการสูญหายของแพ็กเก็ต ดังนั้น การเพิ่ม Delay หรือ Loss จะทำให้ Throughput ลดลงอย่างมีนัยสำคัญ ซึ่งนักศึกษาก็ได้

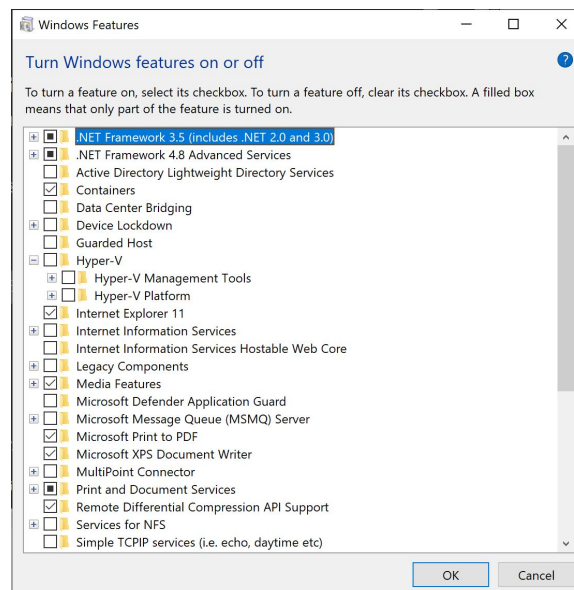
พิสูจน์ความสัณพันธ์นี้ด้วยตนเองในหัวข้อการทดลอง

3 การจัดเตรียมระบบ

การทดสอบอาศัยระบบปฏิบัติการ ลินุกซ์แบบโอเพนซอร์ซ ชื่อว่าอูบันตุ (Ubuntu) เวอร์ชัน 18.04 ขึ้นไป บนระบบปฏิบัติการ วินโดวส์เวอร์ชัน 10 หรือ 11 ทำงานภายใต้ VMware Workstation Player เวอร์ชัน 16 (หรือหากนักศึกษาต้องการใช้ Windows Subsystem for Linux (WSL) สามารถติดตั้งตาม <https://learn.microsoft.com/en-us/windows/wsl/about>

1. อัปเดต ระบบปฏิบัติการวินโดวส์ 10 ให้เป็นแพคเกจล่าสุด (อย่างน้อย 20H1 เดือนพฤษภาคม ค.ศ. 2020)
2. ดาวน์โหลด VMware Workstation Player เวอร์ชัน 16 จากเว็บไซต์ VMware ใช้งานได้ฟรีสำหรับการใช้งานทั่วไป
3. ดาวน์โหลดอูบันตุ (Ubuntu Desktop) อย่างน้อยเวอร์ชัน 18.04

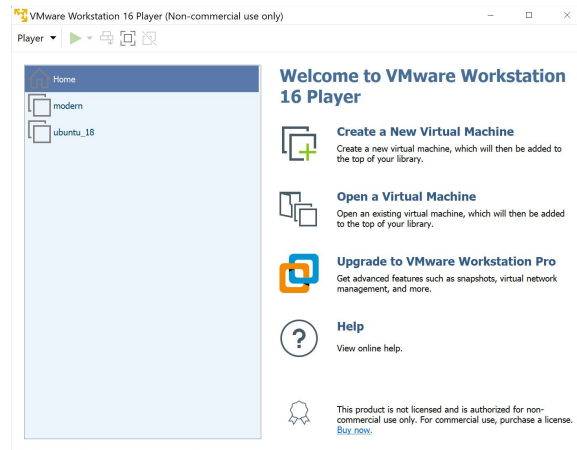
การติดตั้งแนะนำให้ยกเลิกการใช้งาน Hyper-V ที่มากับระบบปฏิบัติการวินโดวส์ โดยไปที่ **Turn Windows features on or off** จากนั้นยกเลิกการใช้งานด้วยการนำ เครื่องหมายถูกด้านหน้า Hyper-V ออก ดังรูปที่ 4 จากนั้นระบบปฏิบัติการวินโดวส์ จะรีสตาร์ทเครื่องใหม่



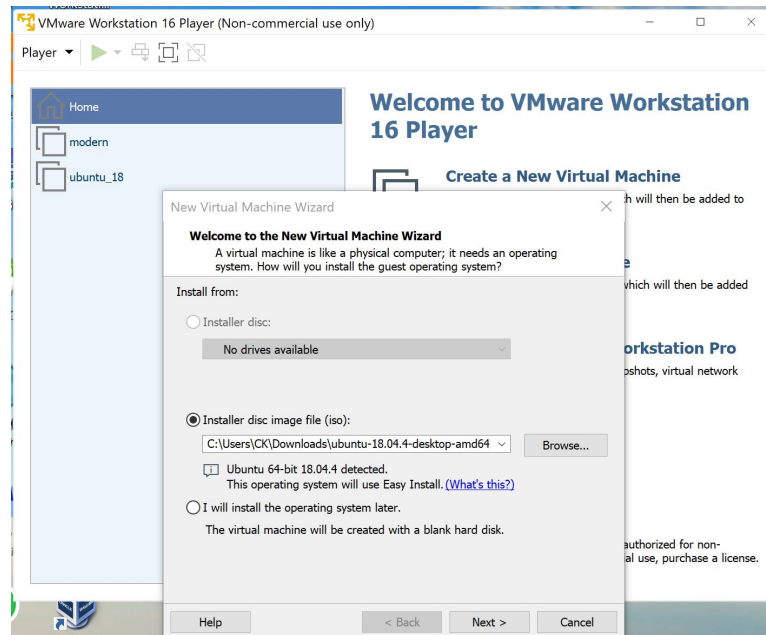
รูปที่ 4: ยกเลิกการใช้งาน Hyper-V

หลังจากติดตั้งโปรแกรม VMware และเข้าโปรแกรม จะปรากฏหน้าจอ ดังรูปที่ 5

จากหน้าจอเริ่มต้น ให้ไปที่ **Create a New Virtual Machine** และกำหนด การติดตั้งอูบันตุจากตำแหน่งที่จัดเก็บ อิมเมจที่ดาวน์โหลดไว้ ดังรูปที่ 6 จากนั้นดำเนินการตามขั้นตอนที่ปรากฏ จะได้ระบบปฏิบัติการ ลินุกซ์ที่ต้องการ



รูปที่ 5: โปรแกรม VMware Workstation Player เวอร์ชัน 16



รูปที่ 6: ติดตั้งอุบ้ันตจากไฟล์อิมเมจที่ดาวน์โหลดไว้

4 การติดตั้งมินิเน็ต

การติดตั้งมินิเน็ตค่อนข้างง่าย ผู้อ่านสามารถโหลดจากเว็บไซต์ mininet.org โดยตรงในรูปแบบเวอร์ชวล แมชชีนแต่ในที่นี้เลือกวิธีการโหลดจากซอร์สโค้ด โดยตรง ซึ่งไม่ยุ่งยากเช่นกัน และจะได้มินิเน็ตเวอร์ชันล่าสุด

```
modern@ubuntu:~$ git clone git://github.com/mininet/mininet
```

เมื่อโหลดเสร็จ ติดตั้งด้วยคำสั่ง `./mininet/util/install.sh [option]` โดย option เป็นเงื่อนไขของการติดตั้ง

- -a: ติดตั้งทั้งหมด ได้แก่ มินิเน็ต, โอเพนวิสวิตช์ (Open vSwitch), โอเพนโฟลว์ (OpenFlow), ไวร์ชาร์ก (Wireshark) และคอนโทรลเลอร์ POX
- -nfv: ติดตั้ง มินิเน็ต, โอเพนโฟลว์และโอเพนวิสวิตช์
- -s mydir: กำหนดตำแหน่งในการติดตั้ง

```
modern@ubuntu:~$ ./mininet/util/install.sh -nfv
Detected Linux distribution: Ubuntu 18.04 bionic amd64
sys.version_info(major=2, minor=7, micro=17, releaselevel=final, serial=0)
Detected Python (python) version 2
Installing Mininet dependencies
[sudo] password for modern:
```

แนะนำให้ใช้ -a เพื่อติดตั้งทั้งหมด เพื่อให้สามารถใช้อโอเพนวิสวิตช์, โอเพนโฟลว์, ไวร์ชาร์กและคอนโทรลเลอร์ POX ได้ในทันที

สำหรับการใช้มินิเน็ตบน Ubuntu 18.04 ให้ติดตั้ง net-tools ด้วย เพื่อใช้คำสั่ง ifconfig

```
modern@ubuntu:~$ sudo apt install net-tools
```

5 การสร้างเน็ตเวิร์คผ่านทางภาพ (Graphical User Interface, GUI)

เพื่อความสะดวก มินิเน็ตจัดเตรียมการใช้งานผ่านทาง GUI เพื่อให้ผู้ใช้สร้าง เน็ตเวิร์คที่ต้องการได้โดยง่าย

```
modern@ubuntu:~/mininet$ sudo examples/miniedit.py
```

จะปรากฏหน้าจอดังรูปที่ 7

ส่วนประกอบสำคัญ ได้แก่

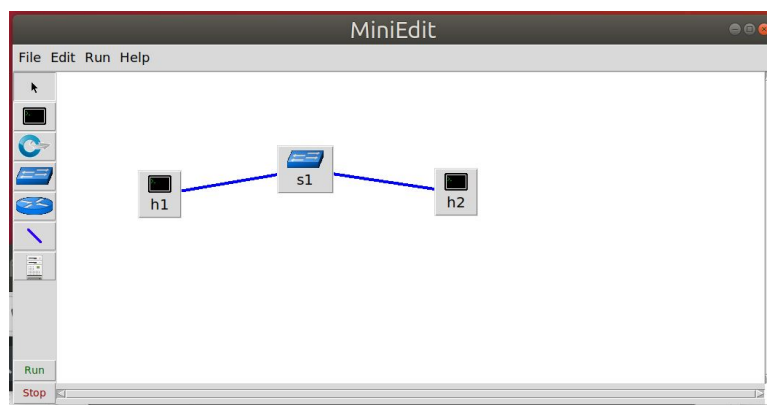
1. **Select** สำหรับเลื่อนหรือลบอุปกรณ์
2. **Host** สำหรับสร้างโฮสต์
3. **Switch** สำหรับใช้งานโอเพนโฟลว์ สวิตช์
4. **Legacy switch** สำหรับสวิตช์ปกติ



รูปที่ 7: หน้าต่าง GUI บนมินิเน็ต

5. **Link** สำหรับเชื่อมต่ออุปกรณ์
6. **Run** เริ่มการทำงานของโปรแกรมเลียนแบบ
7. **Stop** หยุดการทำงานของโปรแกรมเลียนแบบ

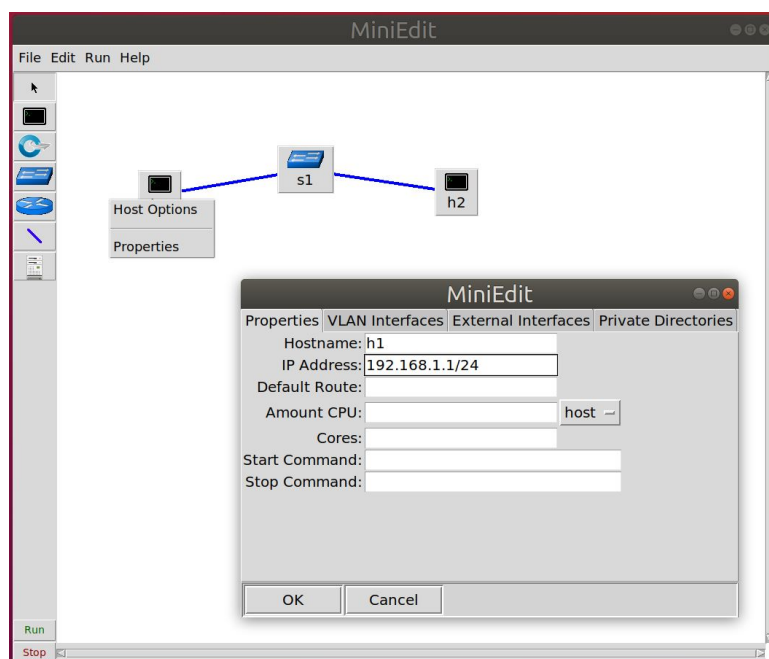
5.1 เน็ตเวิร์คอย่างง่าย



รูปที่ 8: ตัวอย่างการสร้างเน็ตเวิร์คบนมินิเน็ต

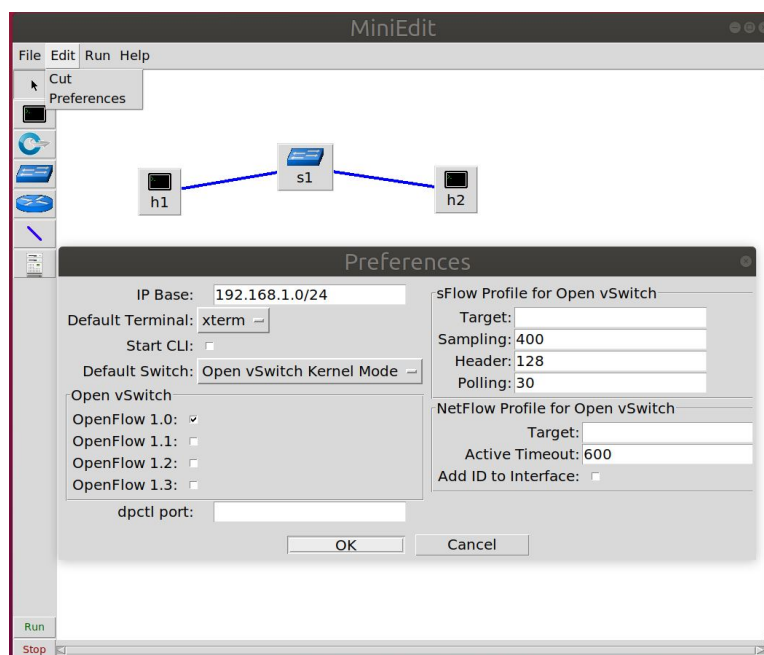
เมื่อสร้างเน็ตเวิร์คดังรูปที่ 8 ผู้อ่านสามารถกำหนด IP address ให้แต่ละโหนดได้ 2 วิธี คือ กำหนดแอดเดรสเอง หรือกำหนดหมายเลขแบบอัตโนมัติ

- การกำหนดแอดเดรสเอง คลิกขวาไปยัง **Properties** ของโหนดนั้น ดังรูปที่ 9



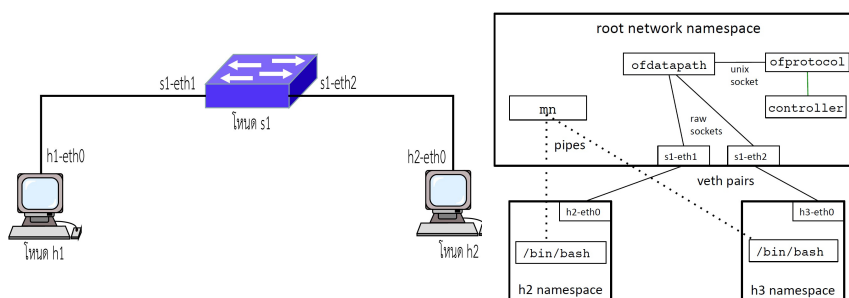
รูปที่ 9: การกำหนดแอดเดรสเอง

- การกำหนดหมายเลขแบบอัตโนมัติ ไปที่เมนู **Edit**→**Preferences** ค่าดีฟอลท์อยู่ที่ 10.0.0.0/8 ให้เปลี่ยนเป็นที่ต้องการ (ในที่นี้กำหนดเป็น 192.168.1.0/24)



รูปที่ 10: การกำหนดหมายเลขแบบอัตโนมัติ

5.2 การเรียกชื่อโฮสต์และสวิตช์



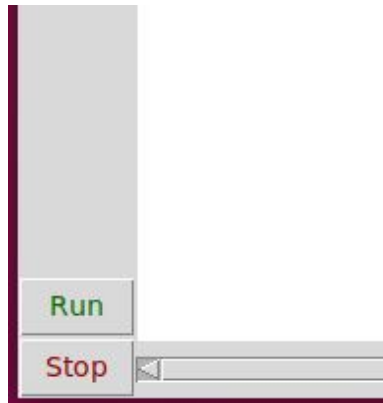
รูปที่ 11: โครงสร้างและการทำงานภายในมินิเน็ต

- โฮสต์ แต่ละโฮสต์ที่สร้างในมินิเน็ตจะมีเนมสเปซของตนเองพร้อม เวอร์ชวล อินเตอร์เฟซโดยทั่วไปใช้ชื่อ h1..hN เช่น โฮสต์ 1 (h1) และโฮสต์ 2 (h2) โดยแต่ละโฮสต์มีอินเตอร์เฟซเป็น h1-eth0 และ h2-eth0 ตามลำดับ การตั้งค่า อินเตอร์เฟซต้องผ่านทางเนมสเปซของโฮสต์นั้นเท่านั้น ดังรูปที่ ??
- สวิตช์ จะถูกกำหนดชื่อ s1..sN โดยอินเตอร์เฟซแรกเป็น s1-eth1 และ อินเตอร์เฟซที่ 2 เป็น s1-eth2 หากมีสวิตช์

ใหม่ (s2) จะเป็น s2-eth1 และ s2-eth2 การเข้าถึงอินเตอร์เฟซของสวิตช์สามารถผ่านทาง "root" namespace ได้ดังรูปที่ ??

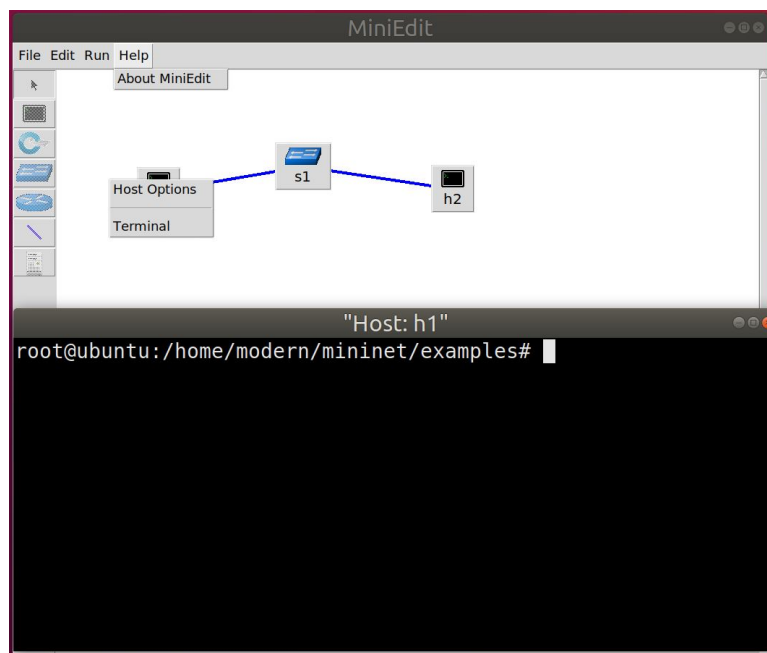
5.3 ทดสอบเน็ตเวิร์คที่สร้างขึ้น

หลังจากสร้างเน็ตเวิร์คที่ต้องการ ให้ไปที่เมนู **Run** หรือคลิกปุ่ม **Run** ดังรูปที่ 12

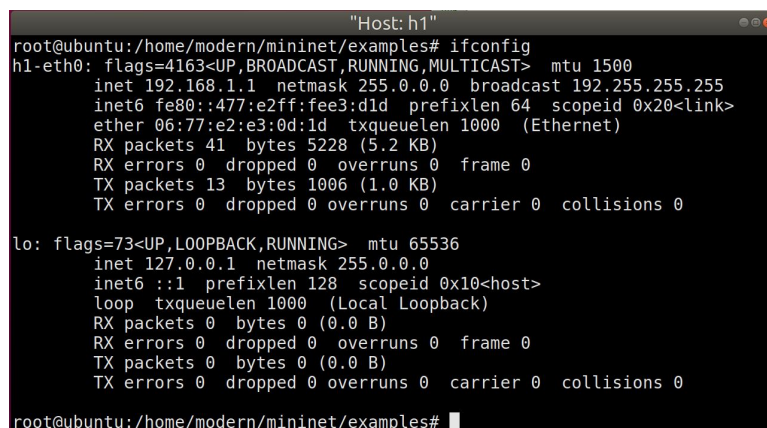


รูปที่ 12: การเริ่มการทำงานของมินิเน็ต

จากนั้นไปที่ host h1 และ h2 เลือก **Terminal** เพื่อเรียกใช้คำสั่งต่าง ๆ รูปที่ 13 แสดง h1 terminal จากนั้นตรวจสอบแอตแตรด้วยคำสั่ง ดังรูปที่ 14



รูปที่ 13: หน้าต่าง Terminal ของ h1



รูปที่ 14: ผลลัพธ์คำสั่ง ifconfig

การตรวจสอบการเชื่อมต่อทำได้โดยใช้คำสั่ง **ping** ตามด้วย IP address ของโหนดปลายทาง (host h2) ดังรูปที่ 15

```
"Host: h1"
root@ubuntu:/home/modern/mininet/examples# ping 192.168.1.2
PING 192.168.1.2 (192.168.1.2) 56(84) bytes of data:
64 bytes from 192.168.1.2: icmp_seq=1 ttl=64 time=0.206 ms
64 bytes from 192.168.1.2: icmp_seq=2 ttl=64 time=0.115 ms
64 bytes from 192.168.1.2: icmp_seq=3 ttl=64 time=0.173 ms
64 bytes from 192.168.1.2: icmp_seq=4 ttl=64 time=0.114 ms
64 bytes from 192.168.1.2: icmp_seq=5 ttl=64 time=0.110 ms
64 bytes from 192.168.1.2: icmp_seq=6 ttl=64 time=0.047 ms
^C
--- 192.168.1.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5126ms
rtt min/avg/max/mdev = 0.047/0.127/0.206/0.051 ms
root@ubuntu:/home/modern/mininet/examples#
```

รูปที่ 15: ผลลัพธ์คำสั่ง ping ระหว่าง h1 และ h2

6 Traffic Control (tc/netem)

Traffic Control (tc) เป็นยูทิลิตี้ของลินุกซ์ที่ใช้กำหนดค่า Packet Scheduler ของ Kernel ทำให้สามารถจำลองสถานะเน็ตเวิร์คที่หลากหลาย เช่น การหน่วงเวลา การสูญหายของแพ็กเก็ต และการจำกัด Bandwidth

```
sudo tc qdisc [add|del|replace|change|show] dev dev_id root netem opts
```

- **sudo:** รันคำสั่งด้วยสิทธิ์ผู้ดูแลระบบ
- **tc:** คำสั่งสำหรับจัดการ NETEM
- **qdisc:** (queue discipline) คือกฎที่กำหนดลำดับการส่งแพ็กเก็ต ที่มาจาก IP protocol output
- **add|del|replace|change|show:** การกระทำต่อ qdisc เช่น add เพื่อเพิ่ม delay, change/del เพื่อเปลี่ยนหรือลบ delay
- **dev_id:** อินเตอร์เฟซที่ต้องการจำลองสถานะ
- **opts:** ค่าของ delay, packet loss, duplication, corruption และอื่น ๆ

6.1 การเพิ่มค่าต่าง ๆ

- **เพิ่ม Delay (add)**

```
sudo tc qdisc add dev h1-eth0 root netem delay 100ms
```

- **sudo:** รันด้วยสิทธิ์ผู้ดูแลระบบ
- **tc:** เรียกใช้ Traffic Control ของลินุกซ์
- **qdisc:** แก้ไข queuing discipline ของ network scheduler

- add: สร้างกฎใหม่
- dev h1-eth0: ระบุอินเตอร์เฟซที่ใช้กฎนี้
- netem: ใช้ Network Emulator
- delay 100ms: เพิ่ม delay 100 มิลลิวินาที

- แก้ไขค่า (change)

```
sudo tc qdisc change dev h1-eth0 root netem delay 50ms
```

- ลบค่าที่ติดตั้ง (del)

```
sudo tc qdisc del dev h1-eth0 root netem
```

- เพิ่มค่า Jitter (add) เพิ่มค่า jitter ± 10 มิลลิวินาที

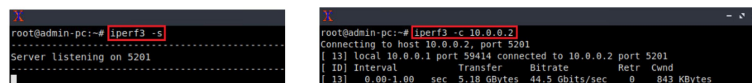
```
sudo tc qdisc add dev h1-eth0 root netem delay 100ms 10ms
```

- เพิ่มค่า Packet Loss (add)

```
sudo tc qdisc add dev h1-eth0 root netem loss 10%
```

7 Iperf3

Iperf3 เป็นเครื่องมือวัดและทดสอบประสิทธิภาพของเครือข่าย โดยเฉพาะความเร็วของ การส่งและรับข้อมูล พัฒนาโดยกลุ่มวิศวกรรมคอมพิวเตอร์ที่มหาวิทยาลัย California เป็นซอฟต์แวร์โอเพนซอร์ส สามารถทดสอบแบบทิศทางเดียวหรือสองทิศทางบน TCP หรือ UDP และกำหนดระยะเวลา ขนาดข้อมูล และพารามิเตอร์อื่น ๆ ได้ตามต้องการ ทำงานในรูปแบบไคลเอนต์ (-c) และเซิร์ฟเวอร์ (-s)



รูปที่ 16: การทดสอบด้วย Iperf3

```
iperf3 [-s|-c] [ options ]
```

- คำสั่งสำหรับเซิร์ฟเวอร์

```
iperf3 -s
```

- คำสั่งสำหรับไคลเอนต์โดยเซิร์ฟเวอร์อยู่ที่ 10.0.0.2

```
iperf3 -c 10.0.0.2
```

- คำสั่งสำหรับไคลเอนต์โดยเซิร์ฟเวอร์อยู่ที่ 10.0.0.2 เป็นเวลา 5 วินาที

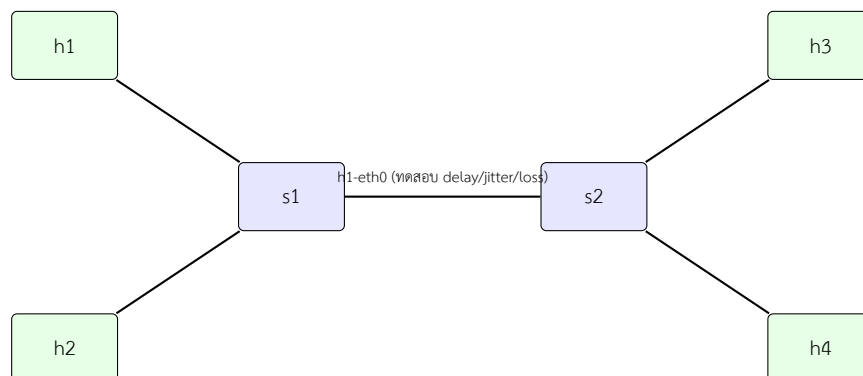
```
iperf3 -c 10.0.0.2 -t 5
```

ผลลัพธ์ที่ได้ประกอบด้วย

- **ID:** หมายเลขการเชื่อมต่อ
- **Interval:** ช่วงเวลาการแสดงผล
- **Transfer:** ปริมาณข้อมูลที่ส่งในช่วงเวลานั้น
- **Bitrate:** ทฤษฎี (throughput) ที่ได้
- **Retr:** จำนวนแพ็กเก็ตที่ส่งใหม่ (retransmission)
- **Cwnd:** ขนาดของ congestion window (เฉพาะ TCP)

8 การทดลอง

จากรูปที่ 17 ให้นักศึกษาสร้างเน็ตเวิร์คและทดสอบการตั้งค่าต่อไปนี้



รูปที่ 17: โทโพโลยีสำหรับการทดลอง

8.1 ทดสอบ Delay

- กำหนด IP address ของเน็ตเวิร์คตามหมายเลขรหัสนักศึกษาของตนเอง
- ตรวจสอบข้อมูลอินเตอร์เฟซของทุกโหนดด้วยคำสั่ง *ifconfig*
- ทดสอบ *ping* จาก h1 ไปยัง h3 บันทึกค่า delay ที่เกิดขึ้น (ค่า min/avg/max/mdev)
- ทดสอบ *ping* จาก h2 ไปยัง h4 บันทึกค่า delay ที่เกิดขึ้น
- เพิ่มค่า delay ที่ h1-eth0 เป็น 100 ms ทดสอบ *ping* จาก h1 ไปยัง h3 บันทึกค่า delay ที่เกิดขึ้น
- ทดสอบ *ping* จาก h1 ไปยัง h3 พร้อมกับ *ping* จาก h2 ไปยัง h3 ในเวลาเดียวกัน เปรียบเทียบผลกับข้อ 8.1
- ลบค่า delay ที่ตั้งไว้ทั้งหมด

คำถาม:

1. ค่า delay พื้นฐาน (ไม่ตั้งค่าใดๆ) ที่วัดได้ระหว่าง h1--h3 ใกล้เคียง 0 ms หรือไม่ เพราะเหตุใด
2. เมื่อเพิ่ม delay 100 ms บน h1-eth0 ค่า RTT ที่วัดได้จาก *ping* ควรมีค่าประมาณเท่าใด (คำนวณและเปรียบเทียบกับผลที่วัดได้จริง)
3. การ ping พร้อมกันจาก h1 และ h2 ไปยัง h3 ทำให้เกิด queuing delay เพิ่มขึ้น หรือไม่ สังเกตได้จากอะไร

8.2 ทดสอบ Jitter

- เพิ่มค่า delay ที่ h1-eth0 เป็น 100 ms และ jitter เป็น 10 ms ทดสอบ *ping* จาก h1 ไปยัง h3 บันทึกค่า delay ที่เกิดขึ้น
- ทดสอบ *ping* จาก h1 ไปยัง h3 พร้อมกับ *ping* จาก h2 ไปยัง h3 เปรียบเทียบผลกับข้อ 8.2
- ลบค่าเดิมทั้งหมด

คำถาม:

1. ค่า mdev (mean deviation) ที่ได้จาก *ping* สัมพันธ์กับค่า jitter ที่กำหนดไว้อย่างไร
2. หากเป็นแอปพลิเคชันแบบเรียลไทม์ เช่น VoIP ค่า jitter ระดับ 10 ms จะส่งผลกระทบต่อคุณภาพเสียงอย่างไร

8.3 ทดสอบ Packet Loss

- เพิ่มค่า loss ที่ h1-eth0 เป็น 5%
- ทดสอบการ ping 150 ครั้ง (`ping <ip address> -c 150`) จาก h1 ไปยัง h3 บันทึกอัตราการสูญหายจริงที่วัดได้
- เปลี่ยนค่าจากข้อ 8.3 เป็น 10% และ 15% เปรียบเทียบผลที่เกิดขึ้น

คำถาม:

1. อัตราการสูญหายจริงที่วัดได้จาก ping ใกล้เคียงกับค่าที่กำหนดไว้ใน `tc/netem` หรือไม่
2. เมื่อ loss เพิ่มจาก 5% เป็น 15% ค่า RTT เปลี่ยนแปลงหรือไม่ เพราะเหตุใด

8.4 ทดสอบด้วย Iperf3

- ทดสอบการสื่อสารระหว่าง h1 และ h2 ด้วย *iperf3* เปรียบเทียบผล กรณีไม่กำหนด delay และกำหนด delay เป็น 100 ms
- เพิ่มค่า jitter ที่ h1 เป็น 10 ms ทดสอบ *iperf3* ระหว่าง h1 และ h2 อีกครั้ง
- เพิ่มค่า loss ที่ h1 เป็น 1% และ 5% ทดสอบ *iperf3* อีกครั้ง ในแต่ละกรณี
- สรุปผลเปรียบเทียบ Transfer, Bitrate และจำนวน Retransmission (Retr) ที่เกิดขึ้นในทุกกรณีในตารางเดียว

คำถาม:

1. จากผลการทดลอง Throughput ลดลงมากที่สุดเมื่อเพิ่ม delay, jitter หรือ loss ตามลำดับ เพราะเหตุใด
2. ผลที่ได้สอดคล้องกับสูตรความสัมพันธ์ระหว่าง Throughput, RTT และ Loss ในรูปที่ 3 หรือไม่ อธิบายเชิงปริมาณ
3. หากทดสอบด้วย UDP (*iperf3 -u*) แทน TCP ผลลัพธ์ในกรณี loss จะ แตกต่างจาก TCP อย่างไร (ทดลองและอธิบาย)

8.5 ติดตั้งอุปกรณ์จริงและทดสอบด้วย Iperf3

- ต่อวงจรเช่นเดียวกับรูปที่ 17 โดยให้เหลือ เพียง 2 เครื่องเท่านั้น (ตามคู่มือของ Lab)
- กำหนด IP Address ของเครื่องตนเองโดยไม่ต้องเชื่อมต่ออินเทอร์เน็ต
- ใช้สวิตช์ในห้องปฏิบัติการทดสอบการสื่อสารด้วย *iperf3* โดย เปรียบเทียบระหว่างสวิตช์ 2 รุ่นที่แตกต่างกัน บันทึกผลว่าแตกต่างกันหรือไม่ อย่างไร

- สลับบทบาทระหว่างไคลเอนต์และเซิร์ฟเวอร์ระหว่างคู่ Lab เปรียบเทียบผลการ ทำงาน ต่างกันหรือไม่ หากต่าง เพราะเหตุใด

คำถาม:

1. ผลที่ได้จากอุปกรณ์จริงแตกต่างจากผลจำลองบนมินิเน็ตหรือไม่ เพราะเหตุใด
2. ปัจจัยใดในอุปกรณ์จริง (เช่น NIC, สาย, driver) ที่อาจทำให้ผลต่างจาก การจำลอง

9 สรุปผลการทดลอง

ให้นักศึกษาสรุปผลการทดลองทั้งหมดในรูปแบบตารางเปรียบเทียบ ดังตัวอย่างในตารางที่ 1 พร้อมอภิปรายความสัมพันธ์ระหว่างพารามิเตอร์ที่ปรับ (Delay, Jitter, Loss) กับผลลัพธ์ที่วัดได้ (RTT, Throughput, Retransmission) โดยอ้างอิงหลักการทางทฤษฎีในหัวข้อที่ 2

การตั้งค่า	RTT (ms)	Throughput (Mbps)	Retr	Loss จริง (%)
ไม่ตั้งค่า (baseline)				
Delay 100 ms				
Delay 100 ms + Jitter 10 ms				
Loss 1%				
Loss 5%				

ตารางที่ 1: ตารางสรุปผลการทดลอง (ให้นักศึกษกรอกผลที่วัดได้)

คำถามสรุป:

1. จากผลการทดลองทั้งหมด พารามิเตอร์ใด (Delay, Jitter, Loss) ส่งผลกระทบ ต่อ Throughput ของ TCP มากที่สุด เพราะเหตุใด
2. หากต้องออกแบบเน็ตเวิร์คสำหรับแอปพลิเคชันที่ไวต่อ Jitter (เช่น VoIP) และแอปพลิเคชันที่ไวต่อ Throughput (เช่น File Transfer) นักศึกษาจะให้ความสำคัญกับพารามิเตอร์ใดเป็นอันดับแรกในแต่ละกรณี
3. จากการทดลองบนอุปกรณ์จริงเทียบกับการจำลองบนมินิเน็ตนักศึกษาคิดว่า มินิเน็ตเหมาะสมสำหรับการศึกษาประสิทธิภาพเครือข่ายในระดับใด และมีข้อจำกัด ใดที่ควรระวังก่อนนำผลไปใช้กับระบบจริง